Vault Access Policy & Key Rotation Specification

Cryptographic Isolation, Automated 24-Hour Rollover, and Failure Recovery

The Guardian Layer Engineering Team

Version 1.0 | November 2026

Abstract

This document defines the access control policies, path-scoped secret isolation, and automated 24-hour asymmetric key rotation protocol governing inter-service communication between the Guardian, Digital Twin, and City Brain layers.

1 Scope and Requirements

The specification enforces mutual cryptographic isolation and automated daily rollover of RSA-4096 communication keys between the `guardian`, `twin`, and `brain` services. All communications require asymmetric signing and encryption, preventing unauthorized lateral movement.

2 Identities and AppRole Policies

Four distinct AppRole identities govern the cryptographic perimeter:

- **guardian-app, twin-app, brain-app:** Scoped microservice identities possessing read-only access restricted strictly to their own cryptographic paths.
- **rotator:** Dedicated administrative AppRole used by the daily rotation daemon. Possesses create/update rights on `secret/data/comm/*` and metadata read rights, but cannot read plaintext key values.

Services authenticate using AppRole credentials to acquire short-lived leased tokens (TTL = 15 min, max_TTL = 1 h). Policies reside in `vault/policies/` and are applied idempotently via `vault/setup_policies.py`.

3 Key Ownership and Mutual Isolation

Each unidirectional channel between layers utilizes an independent RSA-4096 key pair owned by the sender:

- `secret/comm/OWNER/PEER/private` is readable by **OWNER only**.
- `secret/comm/OWNER/PEER/public` is readable by **PEER only**.

A default-deny policy applies to all other paths. Public keys are integrity-protected by Vault (TLS in transit, KV versioning, audit trail) and are never accepted from untrusted channels.

4 Automated 24-Hour Key Rotation Protocol

The key rotation daemon (`vault/rotate_keys.py`) executes daily at 02:00 via a systemd timer. For each of the six communication channels, the workflow executes:

Table 1: Service Key Access Matrix

Service	Readable Private Keys (Signing)	Readable Public Keys (Verification)
Guardian	`guardian/twin`, `guardian/brain`	`twin/guardian`, `brain/guardian`
Twin	`twin/guardian`, `twin/brain`	`guardian/twin`, `brain/twin`
Brain	`brain/guardian`, `brain/twin`	`guardian/brain`, `twin/brain`

1. Generates a fresh 4096-bit RSA key pair.
2. Writes the new public key (creating a new KV version with a unique Key ID `kid`).
3. Writes the new private key stamped with `active_after = now + 1h`.

4.1 Safe Transition Window (Zero Downtime)

- **Dual Verification Window:** Peer receivers query `vault_keys.verification_keys`, accepting both Version N (current) and Version $N-1$ (previous).
- **Delayed Signing Activation:** The sender continues signing with Version $N-1$ until the 1-hour delay expires (`vault_keys.signing_key`), ensuring all peers have cached Version N before first use.
- **Version Retention:** KV v2 retains 5 historical versions before automated pruning.

5 Revocation and Incident Response

- **Compromised Token:** Revoke immediately via `vault token revoke <token>`.
- **Compromised Secret ID:** Destroy via `vault write auth/approle/role/ROLE/secret-id/destroy secret_id=ID` and re-issue.
- **Service Compromise:** Revoke all service tokens, trigger emergency execution of `rotate_keys.py`, and purge compromised key versions via `vault kv destroy -versions=N`.

6 Integration Testing

The security suite in `tests/vault_keys/` validates rotation, AppRole isolation, and rollback mechanisms against a disposable local Vault instance:

```
docker run --rm -d --name vault-test -p 127.0.0.1:8300:8200 \
  -e VAULT_DEV_ROOT_TOKEN_ID=dev-root hashicorp/vault:1.17
export VAULT_TEST_ADDR=http://127.0.0.1:8300 VAULT_TEST_TOKEN=dev-root
uv run pytest tests/vault_keys -v
```